MS-ADM — Divisão de tarefas

Módulo Administrativo · Samantha, Cristiano e Kennedy

Como usar este documento

Cada pessoa tem uma página com as suas tarefas **na ordem em que devem ser feitas**. Marque a caixa quando a tarefa cumprir a definição de pronto (abaixo). A coluna da direita diz o que precisa estar pronto antes (**Espera**) e para quem a tarefa destrava o trabalho (**Libera**). Os números citados na coluna são os da página de cada pessoa.

Cada um é dono de ponta a ponta dos seus módulos: entidade, DTOs (inclusive o **Resumo**, usado na API de integração e nos eventos), serviço, controller, endpoints de integração do módulo, **publicação dos eventos do módulo** e testes. Os códigos entre parênteses (RF, RN, HU, RNF) são os do PDF de escopo.

Quem é dono de quê

	Samantha	Cristiano	Kennedy
Módulos	Especialidade Paciente (com vínculo de convênio)	Médico Segurança e transversais	Convênio e Plano Unidade e Sala Eventos (outbox e RabbitMQ)
Base	Projeto, Docker, Swagger, Actuator, JaCoCo	Erros, códigos ADM-*, validadores CPF/CNPJ	EntidadeBase, Endereco, enum UF, Flyway V1
Fechamento	Cobertura, Postman, apresentação	Revisão de segurança	Dados de exemplo, teste de desempenho

Ordem geral de execução

As fases seguem a ordem em que o trabalho se destrava. Dentro de uma fase os três trabalham ao mesmo tempo; uma tarefa só começa quando o que ela espera já foi entregue.

Fase	Samantha	Cristiano	Kennedy
0 Combinar	Acordos com a turma e entre o trio (lista na página seguinte)		
1 Base	Projeto, Docker, Swagger com o contrato, Actuator, JaCoCo	Erros, códigos ADM-*, @Cpf e @Cnpj	EntidadeBase, Endereco, UF, Flyway V1
2 Fundação	Especialidade (molde dos outros CRUDs)	JWT, auditoria, logs com correlation id	Outbox e RabbitMQ
3 Cadastros	Paciente	Médico	Convênio e Plano, depois Unidade e Sala
4 Ligações	Vínculo paciente–convênio, regra de inativar especialidade	Médico–unidades, Médico–convênios	Inativação em cascata e reativações
5 Integração	Integração de Especialidade e Paciente	Integração do Médico	Integração de Sala, Convênio e Plano
6 Fechamento	Conferir cobertura, Postman, apresentação	Revisão de segurança (RN18, RNF09)	Dados de exemplo, teste de 300 ms

Definição de pronto (vale para toda tarefa)

- Critérios de aceite da história atendidos e testados.
- Endpoint aparecendo no Swagger com exemplo.
- Evento publicado no outbox quando a tarefa cria, altera, inativa ou reativa dado.
- Teste unitário do serviço e teste do controller.
- **Code review de outra pessoa antes do merge**, em rodízio fixo: **Samantha revisa o Cristiano, Cristiano revisa o Kennedy, Kennedy revisa a Samantha.**

Fase 0 — Todos

Antes de escrever código · cada item tem quem puxa a conversa

Estes acordos destravam decisões que aparecem nas páginas de cada um. O responsável puxa a conversa com a turma ou com o outro grupo e registra a decisão.

☐	1	**Stack: PostgreSQL, Flyway, RabbitMQ (ou só REST)** Se a turma fechar outra stack, ajustar as tarefas de Docker, Flyway e eventos.	Puxa: Samantha
☐	2	**Porta do serviço (sugerida 8081) e se vai ter API Gateway**	Puxa: Samantha
☐	3	**Quem emite o token JWT e quais perfis existem** ADMIN, RECEPCAO e token de serviço (RNF09). Se ninguém da turma emitir, o Cristiano faz um emissor simples.	Puxa: Cristiano
☐	4	**Grade de horários do médico fica no MS-AGEND** Confirmar com o grupo do Bruno que o MS-ADM guarda só o cadastro do médico.	Puxa: Cristiano
☐	5	**Preço e tabela do convênio ficam no MS-FATUFINAN**	Puxa: Kennedy
☐	6	**Eventos que cada grupo vai assinar** Usar o catálogo da seção 10 do escopo.	Puxa: Kennedy
☐	7	**Regra de versão da API** Mudança que quebra contrato sobe para /api/v2 e mantém a v1 até os outros migrarem (RNF10).	Todos
☐	8	**Busca de paciente por CPF e nome na URL** GET /pacientes?cpf= conflita com a RNF08 (dado pessoal nunca em URL de listagem). Decidir: aceitar ou trocar por POST /pacientes/busca.	Puxa: Samantha

Samantha

Especialidade · Paciente · Base do projeto · Entrega final

Fase 1 — Base do projeto

☐	1	**Criar o projeto Spring Boot** Spring Initializr com Web, JPA, Validation, PostgreSQL, Flyway, Actuator e AMQP. Pacotes por funcionalidade: comum, paciente, medico, especialidade, unidade, convenio. É a primeira tarefa do projeto inteiro.	Libera: todos
☐	2	**Docker Compose com app, banco e RabbitMQ** Sobe com um único docker compose up (RNF05).	Libera: Kennedy (RabbitMQ, 6)
☐	3	**Swagger com o contrato da seção 9** Todos os endpoints e DTOs aparecendo, mesmo devolvendo dados fixos (RNF02). Mandar o link aos outros grupos: é o que destrava a turma.	Libera: outros grupos
☐	4	**Ativar /actuator/health** Disponível para o gateway e os outros serviços (RNF06).	
☐	5	**Configurar o JaCoCo no build** Relatório de cobertura desde o começo, para todos acompanharem (RNF03).	

Fase 2 — Especialidade (vira o molde dos outros CRUDs)

☐	6	**CRUD de Especialidade** POST, GET, PUT, DELETE em /api/v1/especialidades (RF14, HU07). Nome único sem diferenciar maiúsculas (RN10, ADM-ESP-001). DTOs com EspecialidadeResumo.	Espera: EntidadeBase (Kennedy 1), erros (Cristiano 1) Libera: Cristiano (Médico, 7)
☐	7	**Eventos e reativação de especialidade** Eventos especialidade.criada, atualizada, inativada e reativada no outbox (RN19). PATCH /especialidades/{id}/ativar.	Espera: outbox (Kennedy 4)
☐	8	**Testes de Especialidade** Serviço e controller. Mostrar o padrão para Cristiano e Kennedy copiarem.	

Fase 3 — Paciente (HU01 a HU03)

☐	9	**Entidade Paciente, ResponsavelLegal, enums e DTOs** Campos e validações da seção 7, enums Sexo e Parentesco, repositório e DTOs, incluindo o PacienteResumo com CPF mascarado (usado nos eventos e na integração).	Espera: Endereco e UF (Kennedy 2), @Cpf (Cristiano 3)
☐	10	**Cadastrar paciente (com responsável legal)** POST /pacientes com 201 + Location (RF01, HU01). RN01 CPF válido e único; RN02 data de nascimento; RN03 e RF07 menor de 18 exige responsável com nome, CPF e parentesco (ADM-PAC-003); RN04 celular ou e-mail. Evento paciente.criado.	
☐	11	**Consultar e listar pacientes** GET por id com detalhe completo; GET com filtros nome, CPF e ativo, paginado. Sem filtro de situação traz só ativos; CPF completo traz no máximo um. Listagem com CPF mascarado (RF02, RF03, HU02, RN20).	Espera: decisão da Fase 0 (busca por CPF)
☐	12	**Editar paciente** PUT substitui o recurso. Evento paciente.atualizado com camposAlterados. Trocar CPF para um existente retorna 409 (RF04, HU03).	
☐	13	**Inativar e reativar paciente** DELETE inativa e PATCH /ativar reativa. Continua acessível pelo id com ativo = false (RF05, RN17). Eventos paciente.inativado e reativado.	

☐	14	**Consentimento LGPD** Data e versão do termo; versão obrigatória quando há consentimento (RF26).	
☐	15	**Testes de Paciente** Todos os critérios de aceite de HU01 a HU03.	

Fase 4 — Ligações do Paciente e da Especialidade

☐	16	**Vínculo do paciente com convênio** POST, GET, PUT, DELETE em /pacientes/{id}/convenios (RF06, HU04). RN05 sem dois vínculos ativos no mesmo plano (ADM-PCV-002); RN06 plano e convênio ativos (ADM-PCV-001); RN07 carteirinha não vencida (ADM-PCV-003). Eventos convenio-vinculado, convenio-atualizado e convenio-desvinculado.	Espera: Convênio e Plano (Kennedy 8 e 9)
☐	17	**Vigência do vínculo** isVigente(data) considera a validade e também plano e convênio inativos (RN16: inativar o convênio não mexe no vínculo, mas ele deixa de ser vigente).	Combinar com Kennedy (15)
☐	18	**Bloquear inativar especialidade com médico ativo** Retorna 422 com a quantidade de médicos vinculados (RN11, ADM-ESP-002).	Espera: Médico (Cristiano 7)
☐	19	**Testes do vínculo e da RN11** Critérios de aceite da HU04 e da RN11.	

Fase 5 — API de integração

☐	20	**Especialidades ativas** GET /integracao/especialidades (RF20).	Libera: AGEND, ATEND
☐	21	**Resumo, situação e contato do paciente** GET /integracao/pacientes/{id}, /situacao e /contato. O /situacao devolve 200 com existe e ativo mesmo para id inexistente, em até 300 ms (RF20, RF21, HU10). CPF sempre mascarado (RN20).	Libera: AGEND, ATEND, FATUFINAN, Notificação
☐	22	**Busca de pacientes em lote** GET /integracao/pacientes?ids=a,b,c com no máximo 100 ids (RF22).	
☐	23	**Convênios vigentes na data** GET /integracao/pacientes/{id}/convenios/vigentes?data= (RF24).	Espera: vínculo e vigência (16 e 17)

Fase 6 — Fechamento

☐	24	**Conferir a cobertura** Pelo menos 70% na camada de serviço do projeto todo (RNF03); avisar quem estiver abaixo.	Espera: testes de todos
☐	25	**Coleção do Postman** Uma requisição por endpoint, com exemplos, para a apresentação.	Espera: Fase 5 de todos
☐	26	**Apresentação** Slides e roteiro da demonstração, com uma parte para cada um.	Espera: Postman

Cristiano

Médico · Segurança · Erros e validadores

Fase 1 — Base comum

☐	1	**Tratamento de erros no formato Problem Details** ApiExceptionHandler, NegocioException e NaoEncontradoException; mesmo formato para 400, 404, 409, 422 e 500 (seção 2).	Espera: projeto (Samantha 1) Libera: todos
☐	2	**Catálogo de códigos ADM-*** Códigos e mensagens da seção 9.4: ADM-PAC, PCV, MED, ESP, UNI, SAL, CNV e GEN-404.	Libera: todos
☐	3	**Validadores @Cpf e @Cnpj** Dígitos verificadores, com testes. Usados em Paciente (RN01), Médico, Unidade (RN12) e Convênio.	Libera: Samantha (9), Kennedy (8 e 11)

Fase 2 — Segurança e transversais (enquanto a Especialidade não fica pronta)

☐	4	**Autenticação por JWT** Escrita só com perfil ADMIN ou RECEPCAO; integração por token de serviço (RNF09). Se a turma não tiver emissor, fazer um simples para desenvolvimento.	Espera: decisão da Fase 0
☐	5	**Auditoria** Preencher criadoPor e atualizadoPor com o usuário do token (RF27).	Espera: JWT (4), EntidadeBase (Kennedy 1)
☐	6	**Log estruturado e X-Correlation-Id** Gerar ou repassar o cabeçalho e incluir no log (RNF07).	

Fase 3 — Médico (HU05, HU06)

☐	7	**Entidade Médico e DTOs** Campos da seção 7, repositório, tabela medico_especialidade. Unicidade de CRM + UF (RN08) e CPF. DTOs com MedicoResumo (usado nos eventos e na integração).	Espera: Especialidade (Samantha 6), UF (Kennedy 2)
☐	8	**Cadastrar médico** POST /medicos com 201 (RF08, HU05). Ao menos uma especialidade, todas ativas (RN09, ADM-MED-002); CRM/UF duplicado retorna 409 (ADM-MED-001). Evento medico.criado com as especialidades.	
☐	9	**Consultar, listar e filtrar** GET por id e listagem paginada com filtros nome, CRM, especialidadeId e ativo (RF09). O filtro unidadeId entra na tarefa 13.	
☐	10	**Editar médico e especialidades** PUT substitui o recurso. Evento medico.atualizado (RF10).	
☐	11	**Inativar e reativar médico** DELETE e PATCH /ativar; inativo continua consultável pelo id (RF11, HU06). Evento medico.inativado para o MS-AGEND tratar as consultas futuras.	Libera: Samantha (RN11, 18)
☐	12	**Testes de Médico** Critérios de aceite de HU05 e HU06.	

Fase 4 — Ligações do Médico

☐	13	**Médico nas unidades** unidadeIds no POST e PUT, unidades ativas, e filtro unidadeId na listagem (RF12, prioridade S).	Espera: Unidade (Kennedy 11)

☐	14	**Convênios aceitos** convenioIds no POST e PUT, convênios ativos (RF13, prioridade C).	Espera: Convênio (Kennedy 8)

Fase 5 — API de integração do Médico

☐	15	**Resumo e situação** GET /integracao/medicos/{id} (MedicoResumo) e /situacao, que devolve 200 mesmo se não existir (RF20, RF21).	Libera: AGEND, ATEND, FATUFINAN
☐	16	**Busca em lote e filtros** GET /integracao/medicos?ids=&especialidadeId;=&unidadeId;= (RF22).	Espera: 13
☐	17	**Médico atende especialidade na unidade** GET /integracao/medicos/{id}/atende?especialidadeId=&unidadeId;= devolvendo {atende: true/false} (RF25).	Espera: 13 Libera: AGEND

Fase 6 — Fechamento

☐	18	**Revisão de segurança** Nenhuma rota de integração grava (RN18); toda rota de escrita exige token. Teste automatizado de acesso negado (401/403).	Espera: Fase 5 de todos

Fase 1 — Base comum

☐	1	**EntidadeBase** Classe abstrata com id (UUID), ativo, criadoEm, atualizadoEm, criadoPor, atualizadoPor, inativar() e reativar(). A regra de inativação fica escrita uma vez só (RN17).	Espera: projeto (Samantha 1) Libera: todos
☐	2	**Endereco e enum UF** Embeddable com CEP de 8 dígitos, logradouro, número, complemento, bairro, cidade e UF. Enum UF com as 27 siglas (usado também no CRM do médico).	Libera: Samantha (9), Cristiano (7)
☐	3	**Migração Flyway V1__schema_inicial.sql** As 12 tabelas (8 entidades, 3 associativas e outbox_evento), com chaves, índices únicos e o índice parcial do vínculo ativo. Banco só do MS-ADM (RNF04).	Libera: todos

Fase 2 — Eventos: outbox e RabbitMQ (todos dependem disso)

☐	4	**Gravação do evento na mesma transação** OutboxEvento, OutboxRepository e EnvelopeEvento, com um método simples, por exemplo registrar(tipo, agregadoId, dados, camposAlterados), que cada serviço chama. Cumpre a RN19 e o RF23.	Libera: Samantha (7) e todos os eventos
☐	5	**Publicador agendado** PublicadorOutbox com @Scheduled a cada 5 segundos: publica os pendentes e marca publicado_em.	
☐	6	**Exchange adm.events no RabbitMQ** Tipo topic; routing key igual ao nome do evento. Envelope com eventoId, tipo, versao, ocorridoEm, origem, agregadoId, camposAlterados e dados.	Espera: Docker (Samantha 2)
☐	7	**Avisar o time** Mensagem curta explicando como gravar um evento, com um exemplo.	

Fase 3a — Convênio e Plano (HU09) · primeiro, porque destrava o vínculo do paciente

☐	8	**CRUD de Convênio** POST, GET, PUT, DELETE em /api/v1/convenios (RF18). Registro ANS com 6 dígitos, obrigatório e único (RN15, ADM-CNV-001); CNPJ válido. DTOs com ConvenioResumo. Eventos convenio.criado, atualizado e inativado.	Espera: @Cnpj (Cristiano 3) Libera: Samantha (16), Cristiano (14)
☐	9	**CRUD de Plano** POST e GET em /convenios/{id}/planos; GET, PUT, DELETE em /planos/{id} (RF19). Nome único no convênio; convênio precisa estar ativo. PlanoResumo. Eventos plano.*.	Libera: Samantha (16)
☐	10	**Testes de Convênio e Plano** CRUD e regras RN15 da HU09.	

Fase 3b — Unidade e Sala (HU08)

☐	11	**CRUD de Unidade** POST, GET, PUT, DELETE em /api/v1/unidades (RF15). CNPJ válido e único (RN12, ADM-UNI-001). Eventos unidade.criada, atualizada e inativada.	Libera: Cristiano (13)
☐	12	**CRUD de Sala dentro da unidade** POST e GET em /unidades/{id}/salas; GET, PUT, DELETE em /salas/{id} (RF16). Identificação única na unidade (RN13, ADM-SAL-001); só em unidade ativa (RN14, ADM-SAL-002). SalaResumo. Eventos sala.*.	

☐	13	**Tipo da sala** Enum TipoSala: CONSULTORIO, EXAME, PROCEDIMENTO, OBSERVACAO (RF17).	
☐	14	**Testes de Unidade e Sala** CRUD e regras RN12 e RN13 da HU08.	

Fase 4 — Cascatas e reativações

☐	15	**Inativar convênio em cascata** Inativa os planos e publica convenio.inativado; os vínculos dos pacientes ficam como estão, mas deixam de ser vigentes (RN16, HU09).	Combinar com Samantha (17)
☐	16	**Inativar unidade em cascata** Inativa todas as salas e publica um sala.inativada para cada uma (RN14, HU08).	
☐	17	**Reativar convênio, plano, unidade e sala** PATCH /ativar em cada um, com evento *.reativado. O escopo não lista esses endpoints, mas o RF23 exige evento de reativação.	
☐	18	**Testes das cascatas e reativações** Critérios de aceite restantes de HU08 e HU09.	

Fase 5 — API de integração

☐	19	**Salas** GET /integracao/unidades/{id}/salas?tipo= (só ativas) e /integracao/salas/{id} com SalaResumo (RF20).	Libera: AGEND, ESTOQUE
☐	20	**Convênio e Plano** GET /integracao/convenios/{id} (com planos) e /integracao/planos/{id} (com o convênio).	Libera: FATUFINAN

Fase 6 — Fechamento

☐	21	**Dados de exemplo — V2__dados_exemplo.sql** Especialidades, unidades, salas, convênios, planos, médicos e pacientes, mais uma massa de 10 mil pacientes para o teste de desempenho.	Espera: Fase 4 de todos (schema final)
☐	22	**Teste de desempenho** Endpoints de integração em até 300 ms com 10 mil pacientes, com Postman ou JMeter (RNF01).	Espera: 21 e Fase 5 de todos

Conferência: o escopo inteiro está coberto?

Cada item do PDF de escopo aponta para a pessoa e o número da tarefa (S = Samantha, C = Cristiano, K = Kennedy).

Escopo	O que é	Tarefa
RF01, RF07	Cadastrar paciente e responsável legal	S 10
RF02, RF03	Consultar e listar pacientes	S 11
RF04, RF05	Editar, inativar e reativar paciente	S 12, 13
RF06	Vínculo com convênio	S 16, 17
RF08 a RF11	Médico	C 7 a 11
RF12, RF13	Médico em unidades e convênios	C 13, 14
RF14	Especialidade	S 6, 7
RF15 a RF17	Unidade e Sala	K 11 a 13
RF18, RF19	Convênio e Plano	K 8, 9
RF20, RF21	Integração e situação	S 20, 21 · C 15 · K 19, 20
RF22	Busca em lote	S 22 · C 16
RF23	Eventos	K 4 a 7 + evento em cada CRUD
RF24, RF25	Convênio vigente, médico atende	S 23 · C 17
RF26, RF27	LGPD, auditoria	S 14 · C 5
RN01 a RN04	Paciente	S 10, 12
RN05 a RN07	Vínculo	S 16
RN08, RN09	Médico	C 7, 8
RN10, RN11	Especialidade	S 6, 18
RN12 a RN14	Unidade e Sala	K 11, 12, 16
RN15, RN16	Convênio	K 8, 15 · S 17
RN17, RN19	Inativação lógica, evento após commit	K 1, 4
RN18, RN20	Integração só leitura, CPF mascarado	C 18 · S 9, 21
RNF01	300 ms com 10 mil pacientes	K 22
RNF02, RNF05, RNF06	Swagger, Docker, Actuator	S 3, 2, 4
RNF03	Cobertura 70%	S 5, 24
RNF04	Banco próprio	K 3
RNF07, RNF09	Logs, JWT	C 6, 4
RNF08	LGPD	S 9, 14 · Fase 0
RNF10	Versão da API	Fase 0
Seção 9.4	Códigos de erro	C 2
Etapa 8	Dados de exemplo, Postman	K 21 · S 25

Pontos de atenção

- **Reativação.** O escopo só prevê PATCH /ativar para Paciente e Médico, mas o RF23 pede evento de reativação para todos os dados mestres. Foram incluídas tarefas para os outros (S 7, K 17).
- **RNF08 × busca por CPF.** A busca GET /pacientes?cpf= põe dado pessoal na URL, o que a RNF08 proíbe. Decidir na Fase 0.
- **Resumos antes dos eventos.** O campo dados dos eventos usa o mesmo DTO Resumo da integração; por isso cada Resumo é criado junto com a entidade (Fase 2 e 3), e não só na Fase 5.